

STAR Archive Utility

Version 1.5.0

Technical Architecture, Operating Model and Validation

STAR archive format | Smart analysis | Global deduplication
Additive generations | Transaction safety | Verification

Document status

English technical documentation for version 1.5.0. Product and user-specific names, source paths and machine identifiers have been replaced with neutral terms. Validation results are recorded as observed test outcomes, not universal performance guarantees.

Updated: 1 August 2026

Contents

1. Executive overview
2. Core capabilities and compression modes
3. STAR archive architecture
4. Version 1.5.0 processing model
5. Smart analysis and preparation pipeline
6. Global file deduplication
7. Archive creation workflow
8. ADD generation workflow
9. Browse, extraction and restoration
10. Verification and transaction safety
11. Changes from version 1.4.0
12. Validation results
13. Operational guidance
14. Summary

Scope

This document explains the architecture and observed operating behavior of version 1.5.0. It is not a complete source-code API reference. Timings and storage savings depend on hardware, file count, data composition, cache state, antivirus activity and the selected compression mode.

1. Executive overview

The STAR Archive Utility is a Windows-oriented, self-contained PowerShell archival application. It writes a portable .star file composed of standard TAR-compatible outer records, independently addressable data blocks, a compressed browse catalog and one current canonical manifest.

Version 1.5.0 keeps the integrity and additive-generation model of version 1.4.0, while substantially revising source enumeration, Smart analysis orchestration, build planning, compressor capability probing, threading and the physical build pipeline. The primary objective is to reduce preparatory overhead without removing content analysis, file-level deduplication or verification controls.

Design principle
Analyze once, reuse the sealed catalog, store each unique non-empty payload once, append only new physical data, and verify both physical blocks and logical aliases.

Capability	Version 1.5.0 behavior
Portable single file	Blocks, catalog, metadata and the current manifest are contained in one .star file.
Self-contained deployment	One PowerShell script; embedded C# helpers are compiled at runtime. No separately distributed modules are required.
Smart analysis	Content-aware classification uses file signatures, byte statistics and entropy over bounded samples.
Global deduplication	Byte-identical non-empty files can be represented as direct manifest aliases across paths and ADD generations.
Selective recovery	Browse or extract a file, folder, root, ADD batch or the complete archive.
Verification	Checks block readability, SHA-256, size, path safety, catalog consistency and alias targets.
Transactional ADD	Preserves existing blocks, appends only new physical content and publishes one merged manifest.

2. Core capabilities and compression modes

- Create STAR archives from a file, folder or drive-root source.
- Use Balanced, Smart, Solid or Store mode for newly stored unique data.
- Append timestamped ADD generations without recompressing existing blocks.
- Deduplicate identical non-empty files independently of path, generation and compression mode.
- Create and verify a compressed browse catalog for fast logical navigation.
- Perform full or selected streamed extraction without materializing compressed inner blocks as temporary files.
- Generate persistent text reports for Create, ADD, Extract and Verify operations.

Mode	Behavior	Typical use
Balanced	Mixed groups; XZ9 for compressible data, ZSTD22 for disk-image-like data, STORE for media and archive-like data.	General mixed archives.
Smart	Full content analysis and maximum-compression policy. Unique text, binary, executable, disk-image, archive-like and unknown data use ZSTD22; media remains stored.	When ratio and automated classification matter.

Mode	Behavior	Typical use
Solid	One logical solid data group; XZ9 or ZSTD22 is selected from the measured source profile.	Similar files and simple single-stream storage.
Store	No payload compression. File deduplication, manifest aliases and verification remain active.	Already compressed data, rapid capture and controlled testing.

Important

The selected compression mode applies only to physically stored unique data. Deduplication remains active in every mode, including Store. If an ADD generation contains only duplicates, no new user-data payload is required.

3. STAR archive architecture

```
Outer .star container
  blocks/000001_structure.tar
  blocks/000002_text.tar.zst
  blocks/000003_binary.tar.zst
  star-index.tar.zst
  manifest.json      <- one current final logical entry
```

The outer container separates directory structure, physical payload groups, the browse catalog and the active manifest. Data blocks remain independently addressable and are described by ID, path, source-byte count, file count, compression method, compressed size and SHA-256.

Layer	Responsibility
Outer STAR/TAR	Packages physical blocks, catalog and the final manifest into one file.
Structure block	Stores directories, empty entries and layout information.
Data blocks	Store physically unique payloads using ZSTD22, XZ9 or STORE.
Browse catalog	Compressed logical index used for browse, extraction planning and integrity checks.
Manifest	Describes active blocks, roots, aliases, history, summaries, build mode and canonical tail layout.
Alias map	Maps a logical duplicate path directly to a physically stored target and records its byte length.

Canonical tail

The current manifest is the final logical outer entry. ADD updates are prepared transactionally and publish one merged manifest after structural and incremental validation.

4. Version 1.5.0 processing model

Version 1.5.0 uses a sealed source-catalog model. Source discovery, safe relative-path creation, initial classification and profile aggregation are performed together. Later phases reuse the same catalog instead of independently enumerating the source tree.

```

Enumerate source once
-> build sealed source catalog and profile
-> run Smart content analysis as a batch
-> detect full-file duplicates
-> build unique-file and alias plan
-> create destination-local .stw workroot
-> stage and validate unique files
-> create blocks with one controlled bsdtar process at a time
-> append catalog and manifest
-> Verify

```

Processing property	Version 1.5.0
Source enumeration	Single reusable catalog with explicit excluded-root handling.
Planning records	Primarily retained in memory; final catalog and manifest are published as archive data.
Content analyzer	Embedded C# batch analyzer with bounded parallelism.
Compressor scheduling	Sequential outer scheduling; one active TAR process; compressor-internal threading when supported.
Workroot	Destination-local hidden .stw directory created when physical build begins.
Compatibility fallback	If multithread options fail, the block can be retried in single-thread mode.

5. Smart analysis and preparation pipeline

Smart is the most demanding preparation mode. It performs content analysis before physical block creation and therefore exercises the catalog, analyzer, deduplication planner and group assignment logic.

- File-extension classification establishes an initial group.
- Magic-byte detection recognizes common archive, media, executable and database signatures.
- For non-empty files, up to 1 MiB is sampled from the beginning and end.
- The analyzer calculates entropy, unique-byte count, null-byte count and a conservative text score.
- Analysis is executed in embedded C# with bounded parallelism, avoiding per-byte PowerShell processing.
- The resulting group determines whether unique data enters ZSTD22 or STORE in Smart mode.

Preparation optimization

Version 1.5.0 reduces duplicated source traversal and reuses cataloged paths, metadata and classifications. It also removes the per-item JSONL planning writes used by the earlier pipeline and simplifies stage orchestration. The expensive content reads and SHA-256 confirmations remain in place.

Preparation step	1.4.0 model	1.5.0 model
Enumeration	Several helpers could independently enumerate or profile the source.	One sealed source catalog reused by later phases.
Profile creation	Separate profile-oriented traversal.	Profile accumulated during catalog creation.
Relative paths	Recomputed in multiple planning operations.	Created once and retained with catalog items.
Planning artifacts	Catalog, dedup map and build plan written as JSONL files.	Planning maps and counters primarily held in memory.

Preparation step	1.4.0 model	1.5.0 model
Stage orchestration	Group stages, chunk fallback and parallel block-build coordination.	Simplified sequential one-stage/one-bsdtr physical pipeline.

6. Global file deduplication

Deduplication uses full-file content identity and operates independently of the selected compression mode. Non-empty files are grouped by exact byte length. SHA-256 is calculated only when another candidate has the same length. A confirmed duplicate is omitted from physical data blocks and represented by a direct manifest alias.

Logical path: ADD/ADD_yyyymmdd_hhmmss/Source/example.bin
 Physical target: PrimaryRoot/example.bin
 Manifest record: path + target + bytes

Property	Effect
Path independent	Renaming or moving a duplicate does not prevent deduplication.
Generation independent	A later ADD generation can reuse an earlier physical payload.
Method independent	Balanced, Smart, Solid and Store can reuse the same physical target.
Non-empty threshold	The deduplication threshold is 1 byte; zero-byte files remain ordinary entries.
Direct targets only	Aliases cannot target other aliases; alias chains are rejected.
Build reduction	Only unique payload files enter the physical stage and compressor.

Practical benefit

Store mode still gains content reduction from file deduplication. Smart combines whole-file deduplication first and ZSTD22 compression second, avoiding repeated compression and verification of duplicate payloads.

7. Archive creation workflow

1. Validate the source, destination, available TAR executable and writable locations.
2. Build and seal the source catalog; derive source type and layout policy.
3. Run Smart analysis where required and assign final data groups.
4. Detect duplicates and create direct alias records.
5. Create the hidden destination-local .stw workroot.
6. Stage only physically unique files, preferring hard links and using controlled copy fallback where allowed.
7. Validate the prepared stage against planned paths, sizes, counts and total bytes.
8. Probe STORE, XZ9, ZSTD22 and XZ/ZSTD threading support with real create/extract tests.
9. Create structure and data blocks using explicit TAR format and compression arguments.
10. Build the compressed catalog, write one canonical manifest and publish the final archive.
11. Run full block and alias verification before reporting success.

Threading model

Version 1.5.0 caps compressor threads at the smaller of eight or the logical processor count. Support is proven by a round-trip capability test. Reports record supported algorithms, selected thread count and compatibility retries.

8. ADD generation workflow

Read current manifest and catalog

- > build index from physical unique members only
- > prepare ADD/ADD_yyyymmdd_hhmmss/... logical paths
- > deduplicate against all existing physical files
- > create blocks only for new unique data
- > preserve existing blocks
- > merge browse catalog, roots, history and aliases
- > publish one new canonical manifest
- > incremental validation
- > replace the original only after success

Existing aliases are excluded from the physical deduplication index. Every new alias must target an actual block member, preventing alias-to-alias chains. The build mode is reported as transactional-add and the pipeline as append-new-blocks-preserve-existing.

All-duplicate ADD

When every added non-empty file already exists physically in the archive, the operation adds logical paths and metadata only. The physical archive increase is limited to the updated catalog, manifest and TAR record overhead.

9. Browse, extraction and restoration

Logical archive view

PrimaryRoot/
ADD/
ADD_yyyymmdd_hhmmss/
SourceName/
files and folders

The browse catalog separates the logical recovery view from the physical block layout. A selected path is mapped to the required physical block or, for a duplicate, to its direct physical alias target.

Selection	Result
Complete archive	Restores all content roots and ADD generations.
Primary root	Restores the original tree.
ADD root	Restores additive generations under their isolated timestamped paths.
Timestamped ADD batch	Restores only the selected incremental generation.
Folder or subfolder	Streams only blocks required for the selected descendants.
Single file	Streams the required physical block; alias selections resolve to the physical target.

Selective extraction uses a producer/consumer bridge: the outer TAR process streams one compressed inner block directly to the inner extractor while SHA-256 is calculated in memory. A temporary compressed block file is not required.

10. Verification and transaction safety

- Every manifest block must exist at a safe relative path.
- Each block must be readable and match its recorded compressed size and SHA-256.
- Unsafe, absolute and parent-traversal paths are rejected.
- The browse catalog must match its recorded size and SHA-256 and remain consistent with the manifest.
- Every alias path and target must be safe.
- Every alias target must be a physically stored file, not another alias.
- Alias target size must match the recorded alias byte length.
- The prepared stage must exactly match planned paths, file sizes, counts and total bytes.
- The canonical manifest must remain the final logical outer entry.
- ADD publication must not replace the original archive until the transaction result is valid.

Report state	Meaning
INCREMENTAL OK	The just-completed ADD transaction, updated catalog and manifest passed incremental checks; existing blocks were preserved.
Verification: OK	A separate full Verify checked all listed blocks and all manifest aliases.
Salvage mode	Extraction may skip damaged blocks or aliases and report the omitted content.

11. Changes from version 1.4.0

Area	Version 1.4.0	Version 1.5.0
Source preparation	Multiple helper traversals and file-oriented planning artifacts.	Single reusable sealed source catalog.
Planning persistence	Per-item catalog, dedup and build-plan JSONL writes.	Primarily in-memory planning; final catalog is published into STAR.
Physical build	Parallel independent block build with sequential publication and chunk fallback.	Sequential one-stage/one-bsd tar orchestration with compressor-internal threading.
Zstandard level	ZSTD19.	ZSTD22.
Smart group policy	XZ9 favored for several compressible groups; media and archives commonly stored.	ZSTD22 used for text, binary, executable, disk-image, archive-like and unknown unique data; media remains stored.
Thread probing	Method capability create/extract tests.	Separate round-trip tests for XZ and ZSTD multithread options.
TAR argument construction	Compression arguments embedded in method definitions.	Centralized argument generation with explicit PAX/GNU structure format and threading controls.
Drive-root layout	Archive-root prefix derived from drive label/name.	Virtual display root separated from physically stored content paths.
Browse data	Derived from manifest and block data.	Dedicated compressed browse catalog with recorded SHA-256.
Reporting	Core operation and diagnostic summaries.	Adds duration, scheduler state, active TAR count, threading support, retry

Area	Version 1.4.0	Version 1.5.0
		count, catalog hash and detailed ADD statistics.

Compatibility note

Version 1.5.0 preserves the STAR concept of independent blocks, safe relative paths, manifest aliases and a canonical final manifest. Readers that only support earlier presentation or verification rules may not expose every 1.5.0 function.

12. Validation results

The following results were observed during practical testing on an anonymized Windows workstation. Paths, user names and archive base names are intentionally omitted. These figures validate the tested scenarios only and are not universal benchmarks.

12.1 Large mixed Smart dataset

Metric	Observed result
Logical source size	71.03 GB
Cataloged files	30,083
Unique files physically stored	13,612
Manifest aliases	16,471
Unique source bytes	51.92 GB
Alias bytes removed from physical blocks	19.11 GB
Final archive size	23.93 GB
Overall archive ratio	33.69%
Overall space saved	66.31%
ZSTD22 physical-data ratio	46.09%
Total operation time	04:56:58
Verified blocks	5 of 5
Verified aliases	16,471 of 16,471
Multithread fallback retries	0

The dataset contained mixed text, executable, archive-like and general binary data. Approximately 26.9% of the logical byte volume was represented by aliases, while 54.8% of catalog file entries were duplicate logical copies. ZSTD22 then reduced the 51.92 GB unique payload to 23.93 GB.

12.2 Preparation-phase observation

The hidden destination-local .stw workroot was used as the observable transition from catalog/analysis planning into physical build activity. In the measured 1.5.0 run, the workroot appeared approximately 32 minutes and 5 seconds after the operation started.

Interpretation

The earlier 1.4.0 preparation was observed at more than 40 minutes on the same large mixed workload.

Using 40 minutes as a conservative baseline, 32:05 represents about 19.8% less preparation time. Because the earlier observation was recorded as 40+ minutes rather than an exact timestamp, the result should be reported as approximately 20% or more, not as a precise laboratory percentage.

12.3 Cross-archive ADD deduplication

Metric	Observed result
Added logical source	51.30 MB, 69 files
Cross-archive aliases created	69
New unique user-data bytes	0 B
Previous archive size	14.90 MB
Updated archive size	14.90 MB
Physical increase	3.00 KB
ADD operation time	00:00:08
Incremental validation	INCREMENTAL OK
Subsequent full Verify	4 of 4 blocks; 69 of 69 aliases
Full Verify time	00:00:02

The ADD content was entirely byte-identical to already stored physical payloads. The generation therefore added a new logical root and direct aliases while the archive grew only by catalog, manifest and TAR metadata overhead.

12.4 Validation limits

- The tests do not cover every Windows build, storage device, antivirus configuration or bsdtar implementation.
- Performance is sensitive to file count, random I/O, cache state, same-size deduplication candidates and source changes during analysis.
- A successful benchmark does not replace periodic Verify and independent backup copies.
- Fault injection, unusual reparse points, extreme path lengths and interrupted publication remain separate edge-case test areas.

13. Operational guidance

- Run Verify after important ADD operations and before relying on a STAR archive as the only recovery copy.
- Keep enough free space for the transaction result, workroot, new unique blocks and a safety reserve.
- Do not manually edit manifest.json, the browse catalog or individual blocks inside a STAR archive.
- Keep TXT reports when diagnosing performance or integrity behavior.
- Use Smart when content-aware grouping and ratio matter; use Store when compression is unnecessary but deduplication and verification are still valuable.
- For repeatable preparation benchmarks, use the same source, destination, cache condition, antivirus policy and background load.
- Treat creation of the destination-local .stw workroot as a practical phase marker, not as an exact first-compressed-byte timestamp.
- If an operation is cancelled or interrupted, confirm that the original or resulting archive still passes Verify.

14. Summary

Version 1.5.0 is an evolutionary redesign of a functioning 1.4.0 architecture. It retains portable single-file STAR storage, independent blocks, global full-file deduplication, additive generations, direct aliases, selective streaming and strong verification. The major improvement is the reduction of redundant work before physical compression through a sealed source catalog, combined analysis and profiling, in-memory planning and simplified build orchestration.

The revised compressor layer adds ZSTD22, explicit TAR formats, proven multithread capability detection and single-thread compatibility retry. ADD processing preserves existing blocks and can represent an entirely duplicate generation with metadata-only growth. The observed large mixed Smart test completed with all five blocks and all 16,471 aliases valid, while the preparation phase was approximately 20% shorter than the conservatively stated 1.4.0 baseline.

Overall assessment

Version 1.5.0 is the technically stronger implementation. The largest practical gains are expected in Smart preparation, source catalog reuse, dedup-aware staging, compressor threading, additive updates and diagnostic clarity.

Document status: prepared from the reviewed 1.5.0 implementation, the 1.4.0 technical documentation and anonymized observed validation reports.